

Moisés Montes Iglesias ---- uo296102

[AmpliacionNaves-uo296102-MoisesMontesIglesias.zip](#)

Informe Ampliaciones juego de Naves

Índice:

1. Ampliaciones realizadas
2. Explicación de las ampliaciones
3. Código adjunto

1. Ampliaciones realizadas.

Ampliación	Dificultad
3 Jugador con Vida. El jugador debe tener vidas (inicialmente 3), cuando un enemigo colisiona con el jugador el juego no se reinicia, solo le reduce un punto de vida (y el enemigo se destruye). La vida actual de la nave debe aparecer en pantalla (usar un control de texto o similar) Sí se ha implementado la ampliación (2) los disparos también deben afectar a la vida de la Nave.	5
4 Jugador con disparos finitos: El jugador debe tener un número finito de disparos. Cada vez que elimina a un enemigo recupera un disparo. También debe incluirse un elemento “power up” que al colisionar con el jugador hace que este recupere 10 disparos <i>*En algoritmo de generación de enemigos se deben generar también “power ups” que permitan recuperar disparos cada N iteraciones del juego.</i>	7

6 Monedas: incluir un nuevo tipo de elemento que al colisionar con él nos permita sumar puntos o recuperar vida ((cualquiera de las dos será válida) <i>*En algoritmo de generación de enemigos se deben generar también "monedas" cada N iteraciones del juego.</i>	5
10 Bombas : al colisionar con este elemento se deben eliminar todos los Enemigos que haya en pantalla. <i>*En algoritmo de generación de enemigos se deben generar también "bombas" cada N iteraciones del juego.</i>	6
11 Multijugador local: permitir que dos naves jueguen de manera simultánea, seleccionar utilizar otras teclas para manejar la nave 2.	5

2. Explicación de las ampliaciones

IMPORTANTE PARA TODAS LAS AMPLIACIONES:

Están todas las ampliaciones realizadas comentadas con:
Ampliacion-X (X es el número de la ampliación que hice).

(Espero que estén todas las líneas comentadas y no se me haya olvidado ninguna).

Ampliación 3:

Descripción: Jugador con Vida. El jugador debe tener vidas (inicialmente 3), cuando un enemigo colisiona con el jugador el juego no se reinicia, solo le reduce un punto de vida (y el enemigo se destruye).

La vida actual de la nave debe aparecer en pantalla (usar un control de texto o similar)

Sí se ha implementado la ampliación (2) los disparos también deben afectar a la vida de la Nave.

Como no se ha implementado la ampliación 2, la última parte no la he tenido en cuenta a la hora de implementar la nueva funcionalidad.

Primero, cree una variable que indique la vida del jugador, y cambié que el juego se reiniciase cada vez que colisionase con el enemigo, añadiendo un método que reste una vida al jugador y mientras sea mayor que 0, se elimine al enemigo y nos reste una vida.

La vida actual decidí implementarla en la esquina inferior izquierda para que no moleste a la hora de visualizar el mapeado, y como ocupa zonas donde la nave,

enemigos o elementos a futuro puedan aparecer, la dibujo antes para que no esconda elementos del mapeado.

Código que considero importante:

En la clase Player.cpp:

```
//Ampliacion-3
void Player::loseLife() {
    if (numeroVidas > 0) {
        numeroVidas--;
    }
}
```

En la clase GameLayer.cpp:

```
if (player->isOverlap(enemy)) {
    //Ampliacion-3
    player->loseLife();
    bool eInList = std::find(deleteEnemies.begin(),
        deleteEnemies.end(),
        enemy) != deleteEnemies.end();
    if (!eInList) {
        deleteEnemies.push_back(enemy);
    }
    if (player->numeroVidas <= 0) {
        init();
        return;
    }
    else {
        cargarVidas();
    }
}
```

En la clase GameLayer.cpp:

```
//Ampliacion-3
void GameLayer::cargarVidas() {
    vidas.clear();
    int distancia = 0;
    for (int i = 0; i < player->numeroVidas; i++)
    {
        vidas.push_back(new Vidas(25 + distancia, background-
>fileHeight - 20, game));
        distancia += 50;
    }
}
```

Ampliación 4:

Descripción: Jugador con disparos finitos: El jugador debe tener un número finito de disparos. Cada vez que elimina a un enemigo recupera un disparo.

También debe incluirse un elemento “power up” que al colisionar con el jugador hace que este recupere 10 disparos

**En algoritmo de generación de enemigos se deben generar también “power ups” que permitan recuperar disparos cada N iteraciones del juego*

Cree una variable con una cantidad de disparos inicial de 3, el elemento power up, como no encontré ninguna imagen de power up en la carpeta res, escogí la imagen planeta_2 que es la siguiente:



Cuando el player colisione con este elemento ganará 10 disparos adicionales a los que ya tiene.

Código que considero importante:

En la clase GameLayer.cpp:

```
//Ampliacion-4
if (player->disparos != 0) {
    for (auto const& projectile : projectiles) {
        projectile->draw();
    }
}
```

En la clase GameLayer.cpp:

```
//Ampliacion-4
if (newProjectile != NULL && player->disparos != 0) {
    projectiles.push_back(newProjectile);
    player->disparos--;
}
```

Ampliación 6:

Descripción: Monedas: incluir un nuevo tipo de elemento que al colisionar con él nos permita sumar puntos o recuperar vida ((cualquiera de las dos será válida)

**En algoritmo de generación de enemigos se deben generar también “monedas” cada N iteraciones del juego.*

La implementación de las monedas fue similar a la de los enemigos, cuando el player colisiona con esta, si mantiene sus 3 vidas, se sumará un punto al contador, si el player tiene 1 o 2 vidas, se le recuperará una.

Cree una clase moneda, que genera monedas cada N iteraciones, y estas se mueven en la misma dirección que las naves.

La generación de las monedas cogí un número aleatorio mayor que la de las naves.

Código que considero importante:

En la clase GameLayer.cpp:

```
//Ampliacion-6
//Colisiones con monedas
for (auto const& moneda : monedas) {
    if (player->isoverlap(moneda)) {
        if (player->numerovidas < 3) {
            player->numerovidas++;
            cargarVidas();
        }
    }
}
```

```

    }
    else {
        points++;
        textPoints->content = to_string(points);
    }
}

//Ampliación-6
//Generar monedas
newMonedaTime--;
if (newMonedaTime <= 0) {
    int rX = (rand() % (600 - 500)) + 1 + 500;
    int rY = (rand() % (300 - 60)) + 1 + 60;
    monedas.push_back(new Monedas(rX, rY, game));
    newMonedaTime = 620;
}

```

Ampliación 10:

Descripción: Bombas: al colisionar con este elemento se deben eliminar todos los Enemigos que haya en pantalla.

**En algoritmo de generación de enemigos se deben generar también “bombas” cada N iteraciones del juego.*

La creación de esta ampliación es igual que la moneda, cambiando solo la función que hace el elemento al entrar en contacto con el player.

El elemento bomba, como no encontré ninguna imagen de bomba en la carpeta res, escogí la imagen planeta_1 que es la siguiente:



Cuando el player entre en contacto con esto, eliminará las naves que haya en el mapa.

Código que considero importante:

En la clase GameLayer.cpp:

```

//Ampliacion-10
list<Bombas*> deleteBombas;
//Colisiones con bombas
for (auto const& bomba : bombas) {
    if (player->isOverlap(bomba)) {
        enemies.clear();
    }
}

```

Ampliación 11:

Descripción: Multijugador local: permitir que dos naves jueguen de manera simultánea, seleccionar utilizar otras teclas para manejar la nave 2.

Para crear al segundo jugador, cree un nuevo Player en la clase GameLayer.h llamado player2. Los mismos métodos en los que participaba el player1, pude hacerlos en player 2

Código que considero importante:

En la clase GameLayer.cpp:

```
case SDLK_RIGHT: // derecha
    controlMoveXPlayer2 = 1;
    break;
case SDLK_LEFT: // izquierda
    controlMoveXPlayer2 = -1;
    break;
case SDLK_UP: // arriba
    controlMoveYPlayer2 = -1;
    break;
case SDLK_DOWN: // abajo
    controlMoveYPlayer2 = 1;
    break;
case SDLK_RCTRL: // dispara
    controlShootPlayer2 = true;
    break;
}
case SDLK_RIGHT: // derecha
    if (controlMoveXPlayer2 == 1) {
        controlMoveXPlayer2 = 0;
    }
    break;
case SDLK_LEFT: // izquierda
    if (controlMoveXPlayer2 == -1) {
        controlMoveXPlayer2 = 0;
    }
    break;
case SDLK_UP: // arriba
    if (controlMoveYPlayer2 == -1) {
        controlMoveYPlayer2 = 0;
    }
    break;
case SDLK_DOWN: // abajo
    if (controlMoveYPlayer2 == 1) {
        controlMoveYPlayer2 = 0;
    }
    break;
case SDLK_RCTRL: // dispara
    controlShootPlayer2 = false;
    break;
}
```